

Web Service Design and Practice

10. WebRTC

Dr. Kyudong Park

School of Information Convergence



Contents

- 기존 코드 수정
 - Front
 - 미디어 처리
- WebRTC
- 미디어 출력
- 모바일 및 외부 접속

기존 소스코드의 수정

- HTML 코드 수정
 - 카메라 요소 / 제어 버튼

```
<video id="myFace" autoplay playsinline></video>
<button id="mute">Mute</button>
<button id="camera">Turn Camera Off</button>
```

- javascript 추가
 - client.js
 - 카메라 및 마이크 제어

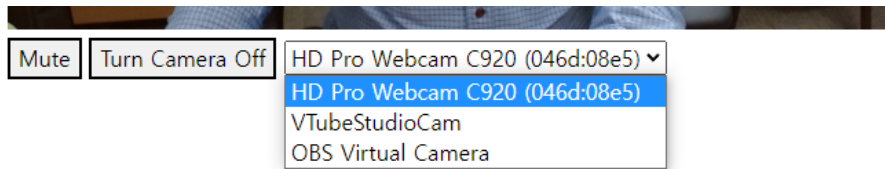
```
▼ [MediaStreamTrack] ⓘ
  ▼ 0: MediaStreamTrack
    contentHint: ""
    enabled: true
    id: "8aa30426-2d5f-4c51-88f9-5b3eafe90c2b"
    kind: "video"
    label: "HD Pro Webcam C920 (046d:08e5)"
    muted: false
    oncapturehandlechange: null
    onended: null
    onmute: null
    onunmute: null
    readyState: "live"
    ▶ [[Prototype]]: MediaStreamTrack
  length: 1
  ▶ [[Prototype]]: Array(0)
```

[client.js:27](#)

```
▼ [MediaStreamTrack] ⓘ
  ▼ 0: MediaStreamTrack
    contentHint: ""
    enabled: true
    id: "aae9b277-5296-4638-b60c-b9cedec9660"
    kind: "audio"
    label: "마이크(Yeti Stereo Microphone) (046d:0ab7)"
    muted: false
    oncapturehandlechange: null
    onended: null
    onmute: null
    onunmute: null
    readyState: "live"
    ▶ [[Prototype]]: MediaStreamTrack
  length: 1
  ▶ [[Prototype]]: Array(0)
```

클라이언트 기능 추가

- 카메라 목록 표시하기
 - select 활용
- client.js 에서 카메라 목록 받아와서 표시해주기



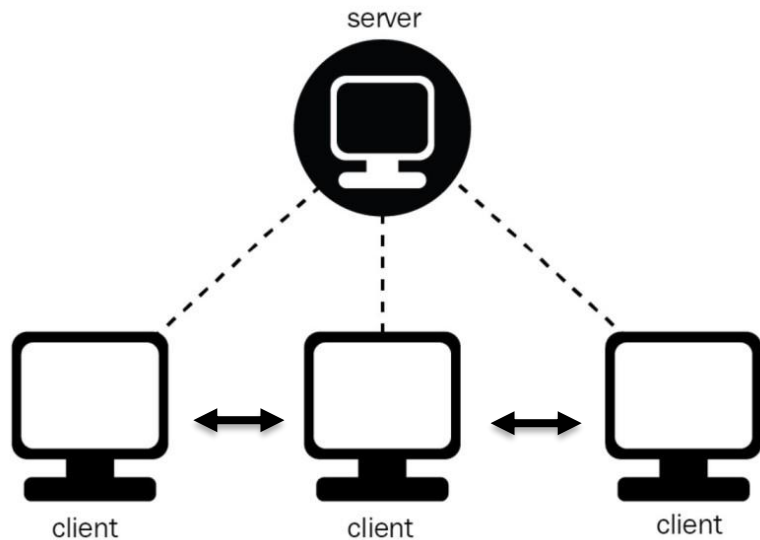
- 선택 시 device 변경하기
- 사용 중인 카메라 표시하기

채팅방 기능 추가

- 채팅방 이름을 통해서 room 에 접속할 수 있도록 추가
- 접속한 이후 카메라 등 미디어 켜도록 수정
- 채팅방 이름 저장 및 전송
 - 동일 채팅방 접속 시, 이벤트 발송 테스트
 - `socket.join(roomName)`
 - `socket.to(roomName).emit()`

WebRTC

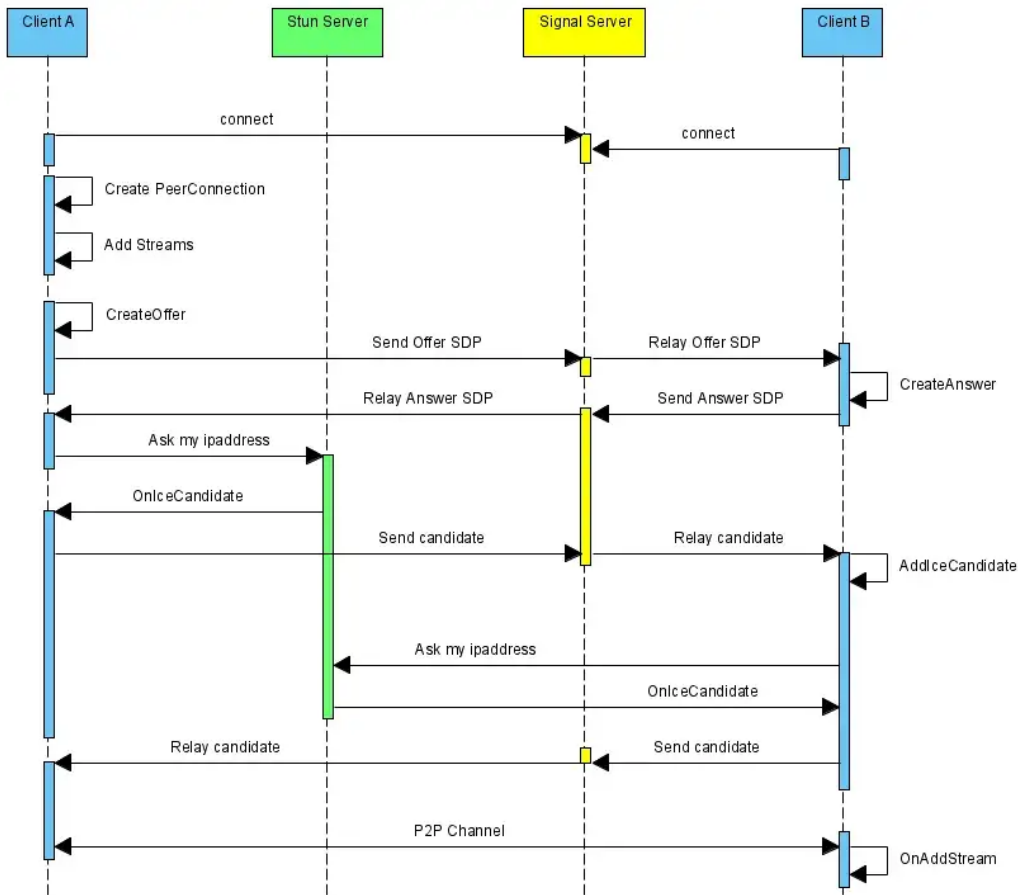
- WebRTC
 - Web Real-Time Communication
 - peer-to-peer 로 통신 가능
 - 서버가 전혀 필요 없는 것은 아님 (Signaling을 통해서 필요한 최소한의 정보를 주고 받음)



WebRTC

• 접속 프로세스

- Offer
- Answer
- ICE Candidate
 - internet connectivity establishment
- P2P 접속



미디어 교환

- 상대의 미디어 출력

```
const handleAddStream = (data) => {  
  const peerFace = document.getElementById("peerFace");  
  peerFace.srcObject = data.stream;  
};
```

- 카메라 변경 처리

```
const handleCameraChange = async () => {  
  //console.log(cameraSelect.value);  
  await getMedia(cameraSelect.value);  
  
  if (myPeerConnection) {  
    const videoTrack = myStream.getVideoTracks()[0];  
    const videoSender = myPeerConnection  
      .getSenders()  
      .find((sender) => sender.track.kind === "video");  
    videoSender.replaceTrack(videoTrack);  
  }  
};
```


모바일과 테스트 해보기 (local tunnel)

- local tunnel
 - 로컬 서버를 외부에서도 접근할 수 있게 설정하는 모듈
 - `$ npm i -g localtunnel`
 - `lt` 명령어 활용가능
 - 로컬 서버가 동작 중일 때, 새로운 터미널에서
 - `$ lt --port 3000`

공유기 환경과 독립

- 동일 wifi 내 에서만 정상 동작함
 - STUN 서버가 필요함
 - 공용IP 주소를 찾아줌
- 구글에서 제공하는 STUN 서버 활용

```
const makeConnection = () => {  
  myPeerConnection = new RTCPeerConnection({  
    iceServers: [  
      {  
        urls: [  
          'stun:stun.l.google.com:19302',  
          'stun:stun1.l.google.com:19302',  
          'stun:stun2.l.google.com:19302',  
          'stun:stun3.l.google.com:19302',  
          'stun:stun4.l.google.com:19302',  
        ],  
      },  
    ],  
  });  
};
```

Advanced

- 채팅 기능과 섞어보기
- UI 꾸며보기
- 접속 종료 시에 대한 처리 구현해보기
- 영상 스트림 받아와서 배경 제거 해보기
 - mediapipe 등 활용
- 배포
 - HTTPS 필수!

Thank you

Q & A